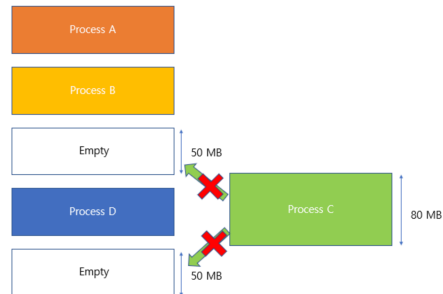


CS 스터디 발표

외부단편화

- 가변 크기 할당 시, 전체 빈 공간은 충분하지만 **연속된 큰 공간이 없어서** 못 쓰는 경우

예: 1MB free, 하지만 256KB가 여기저기 쪼개져 있으면 512KB 요청은 못 받음



내부단편화

- 블록을 고정 크기로 나누었는데, 마지막에 다 안 채워서 낭비되는 공간

예: 1KB 블록에 900B만 쓰면 100B가 낭비됨



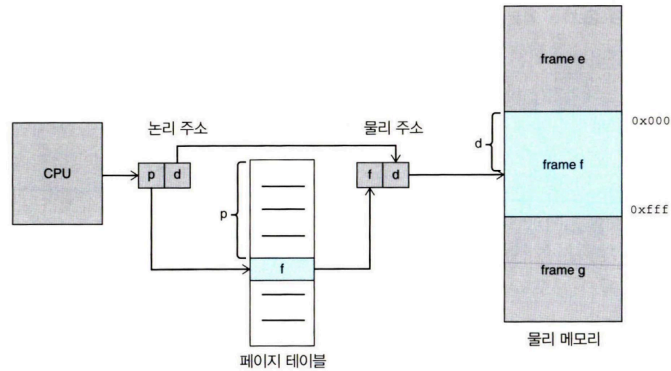
페이징 기법

물리 메모리 → 프레임이라 불리는 같은 크기 블록으로 나눔

논리 메모리 → 페이지라 불리는 같은 크기 블록으로 나눔

CPU의 모든 주소는 **<페이지 번호(p) + 페이지 오프셋(d)>** 으로 나뉘짐

- 페이지 번호 : 페이지 테이블을 액세스할때 사용돼
- 페이지 오프셋 : 참조되는 프레임안에서의 위치



여기서 프레임의 크기와 마찬가지로 페이지 크기는 하드웨어에 의해 정해짐

페이지 크기는 2의 거듭제곱, 보통 페이지당 4KB~1GB 사이

페이지 크기로 2의 거듭제곱을 선택하면 논리 주소를 페이지 번호와 페이지 오프셋으로 쉽게 변환가능함

[페이징 기법으로 논리 주소를 물리주소로 매핑하는 방법]

가상 주소 = (페이지 번호, 오프셋)

변환 과정

1. CPU가 페이지 번호를 가져옴
2. 페이지 테이블에서 해당 페이지 번호 → 프레임 번호 찾음
3. 실제 주소 = (프레임 번호 × 페이지 크기) + 오프셋

예시

- 페이지 크기 = 1KB
- 가상 주소 = (페이지 3, 오프셋 100)
- 페이지 테이블: 페이지 3 → 프레임 7
- 실제 주소 = $7 \times 1024 + 100 = 7268$

[페이징 기법의 장단점]

외부단편화 해결가능 하지만 내부 단편화 발생(왜냐면 할당은 항상 프레임의 정수배로 할당되기 때문)

ex) 2048B가 페이지 크기이고 프로세스가 사용해야하는 메모리 72766B라면 35개 페이지 할당 36번째 할당하는데 $2048 - 1086 = 962B$ 내부 단편화 발생

세그멘테이션 기법

- 가상 메모리를 서로 크기가 다른 논리적 단위로 분할한 것을 의미함
- 프로세스를 물리적 단위인 페이지가 아닌 논리적 단위인 세그먼트로 분할해서 메모리에 적재하는 방식임
- 세그먼트는 의미가 같지 않는 논리적 내용을 기준으로 프로그램을 분할하기 때문에 크기가 같지 않음

세그먼트 방식 동작과정

- 분할 방식을 제외하고 페이징과 세그멘테이션이 동일하기 때문에 매핑 테이블의 동작 방식도 동일함
- 단, 논리 주소의 앞 비트들은 페이징 번호가 아닌 세그먼트 번호가 됨

`<segment, offset>` 형태로 구성되고 세그먼트 번호를 통해 세그먼트의 기준(세그먼트의 시작 물리 주소)와 한계(세그먼트의 길이)를 파악할 수 있음

세그멘테이션 장단점

장점 : 내부 단편화 문제가 해소됨, 보호와 공유 기능을 수행할 수 있음, 프로그램의 중요한 부분과 중요하지 않은 부분을 분리하여 저장할 수 있고 같은 코드 영역은 한 번에 저장할 수 있음

단점 : 외부 단편화 문제가 생길 수 있음

페이징과 세그멘테이션 차이 한눈에 알아보기

구분	세그멘테이션	페이징
단위	논리적 의미 단위	고정 크기 블록
크기	가변	고정
단편화	외부 단편화	내부 단편화
주소 구조	(세그먼트 번호, 오프셋)	(페이지 번호, 오프셋)

어떤 주소공간이 있을 때, 이 공간이 수정 가능한지 확인할 수 있는 방법이 있나요?

세그멘테이션에서의 권한 확인

주소 변환 과정

1. CPU가 논리 주소 (세그먼트 번호, 오프셋) 을 생성
2. 세그먼트 테이블에서 **Base**(시작 주소), **Limit**(크기), ****권한 비트(r/w/x)****를 확인
3. 오프셋이 Limit보다 작은지 확인하고, 권한 비트에 맞는 접근인지 체크
4. 문제가 없으면 물리 주소 = Base + 오프셋

[세그먼트 테이블]

세그먼트 번호	Base	Limit	권한(rwx)
0: 코드	1000	400	r-x
1: 데이터	2000	600	rw-
2: 스택	3000	200	rw-

예: 세그먼트 0(코드) 영역에 쓰기(write) 요청 시 → 권한(r-x) 에 w가 없으므로 **fault 발생**

페이징에서의 권한 확인

주소 변환 과정

1. CPU가 논리 주소 (페이지 번호, 페이지 오프셋) 생성
2. 페이지 테이블에서 페이지 번호를 인덱스로 접근
3. 페이지 테이블 엔트리(PTE)의 **프레임 번호 + 권한 비트(r/w/x)** 확인
4. 권한 비트 허용 시 접근 가능

[페이지 테이블]

페이지 번호	프레임 번호	권한(rwx)
0	5	rw-
1	9	r--
2	2	rwX

예: 페이지 1은 권한이 r--라서 읽기만 가능

쓰기 시도 → **페이지 폴트(Page Fault)** 발생

정리

- Linux: `/proc/<pid>/maps` → 프로세스 메모리 구역별 권한 확인 가능
- Windows: `VirtualQuery()` API로 메모리 권한 조회 가능

32비트에서, 페이지의 크기가 1kb 이라면 페이지 테이블의 최대 크기는 몇 개일까요?

[주어진 상황]

32비트 주소 공간 = $2^{32} = 4GB$

페이지 크기 = $1KB = 2^{10}$

필요한 페이지 수 = 전체 주소 공간 ÷ 페이지 크기

→ $2^{32} \div 2^{10} = 2^{22} = 4,194,304$ (약 420만 개)

즉, 최대 **4,194,304개 페이지**를 관리해야 하며, 페이지 테이블 엔트리도 그만큼 필요하다.

(참고)

페이지 테이블 엔트리(PTE) 개수

- 각 페이지마다 **페이지 테이블 엔트리 1개**가 필요
- 따라서 페이지 수 = 페이지 테이블 엔트리 수
- 즉, 최대 **4,194,304개의 PTE** 필요

페이지 테이블 크기 계산 예시

- 보통 하나의 PTE 크기는 4바이트(32비트 시스템 기준)
- 전체 페이지 테이블 크기 = $4,194,304 \times 4B = 16MB$
- 즉, 하나의 프로세스가 전 주소 공간을 다 쓰려면 페이지 테이블만 해도 16MB 필요

(Deep Dive) 실제 운영체제는 이 문제를 해결하기 위해 **다단계 페이지 테이블** 방식

32비트 운영체제는 램을 최대 4G 까지 사용할 수 있습니다. 이 이유를 페이징과 연관 지어서 설명해 주세요.

32비트 CPU → 주소 레지스터가 32bit → 표현할 수 있는 주소의 최대 크기는 $2^{32} = 4GB$

페이징 구조에서는 **가상 주소 공간 = 페이지 번호 + 오프셋**

페이지 번호가 아무리 많아도, 전체 가상 주소 공간 크기는 4GB를 넘을 수 없음

따라서 32비트 운영체제에서 하나의 프로세스가 접근할 수 있는 최대 메모리 공간은 **4GB**

요약: **32비트 주소 버스 한계(2^{32}) 때문에 페이징으로 나뉘도 최대 4GB까지만 관리 가능하다.**

C/C++ 개발을 하게 되면 Segmentation Fault 라는 에러를 접할 수 있을텐데, 이 에러는 세그멘테이션/페이징과 어떤 관계가 있을까요?

- **세그멘테이션 관점**
 - 세그먼트 테이블에서 허용되지 않은 접근(r/w/x 위반, 오프셋 범위 초과) → Fault 발생
- **페이징 관점**
 - 페이지 테이블에 매핑되지 않은 영역 접근 (invalid page)
 - 쓰기 권한 없는 페이지에 write 요청
 - 페이지 폴트(Page Fault) 발생 → OS에서 처리 불가능 → Segmentation Fault로 종료

즉, **Segmentation Fault**는 세그멘테이션/페이징의 보호 비트 검사 과정에서 발생하는 접근 위반 오류

(Deep Dive)

기본 흐름

1. C/C++ 코드가 어떤 포인터로 메모리에 접근(읽기/쓰기/실행) 시도
2. **MMU**(메모리 관리 장치)가 주소 변환 + **권한 비트 체크**
 - 페이징: **PTE(페이지 테이블 엔트리)**의 **R/W/X** (읽기/쓰기/실행), **P(존재)**, **U/S(유저/커널)** 플래그 확인
 - (세그멘테이션 사용 시) **세그먼트 디스크립터의 Base/Limit(범위)**와 **r/w/x** 확인
3. 위반이면 CPU가 **예외**(x86의 경우 #PF: Page Fault 등) 발생
4. 커널의 예외 핸들러가 상황을 복구할 수 없으면 **SIGSEGV**(또는 유사 예외)를 프로세스에 전달 → 프로세스 종료(+ 코어덌프)

리눅스에서 흔한 SIGSEGV 코드

- `SEGV_MAPERR` : **아예 매핑되지 않은 주소 접근** (예: `NULL` (0) 또는 완전 엉뚱한 주소)
- `SEGV_ACCERR` : **매핑은 됐지만 권한 위반** (예: 읽기 전용 페이지에 쓰기, 실행 금지 페이지에서 실행)

현대 리눅스/윈도우는 **페이징 기반 보호**가 중심

아래와 같은 PTE 플래그 위반이 곧바로 **SegFault**로 이어짐

- **존재 비트(P)가 0**: 해당 가상 페이지가 물리 메모리에 **매핑되어 있지 않음**
 - 예) `int *p = (int*)0x1; *p = 42;` → 대부분 `SEGV_MAPERR`
- **쓰기 금지 페이지에 쓰기 (R/W 위반)**
 - 예) `.text` (코드)나 `.rodata` (읽기전용 데이터)에 쓰기 시도 → `SEGV_ACCERR`
- **실행 금지(NX, Execute Disable) 페이지에서 실행 (X 위반)**
 - 예) 힙/스택(보통 non-exec) 메모리에 임의 바이트를 써놓고 함수포인터로 점프
- **유저 모드가 커널 전용 페이지 접근 (U/S 위반)**